Lecture No 2 .
Unified Modeling Language (UML)

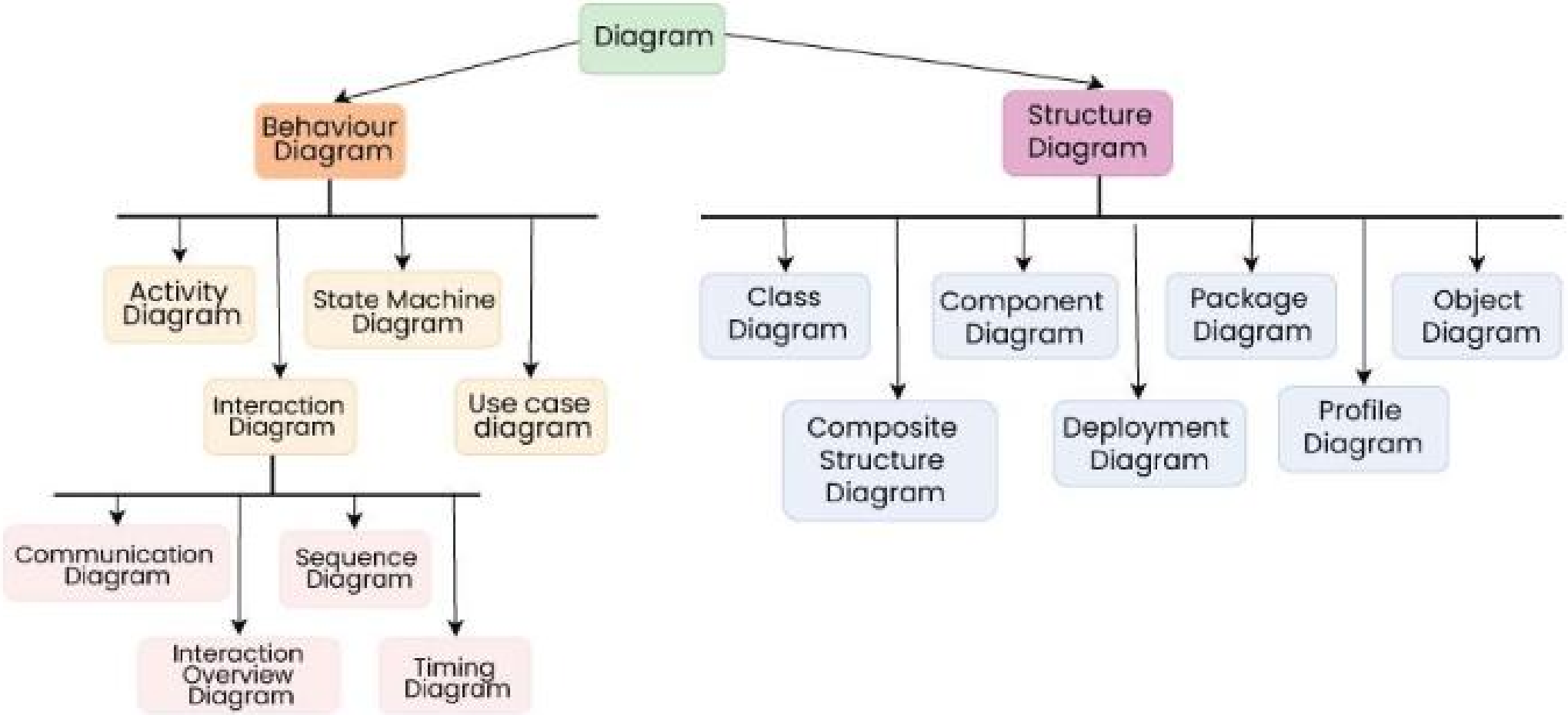
Unified Modeling Language (UML) Diagrams

- Unified Modeling Language (UML) is a general-purpose modeling language.
- The main aim of UML is to define a standard way to visualize the way a system has been designed. It is quite similar to blueprints used in other fields of engineering.
- UML is not a programming language, it is rather a visual language.
- We use UML diagrams to show the behavior and structure of a system.
- UML helps software engineers, businessmen, and system architects with modeling, design, and analysis.

Why do we need UML?

- Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate amongst them.
- Businessmen do not understand code. So UML becomes essential to communicate with non-programmers about essential requirements, functionalities, and processes of the system.
- A lot of time is saved down the line when teams can visualize processes, user interactions, and the static structure of the system.

Types of UML Diagrams

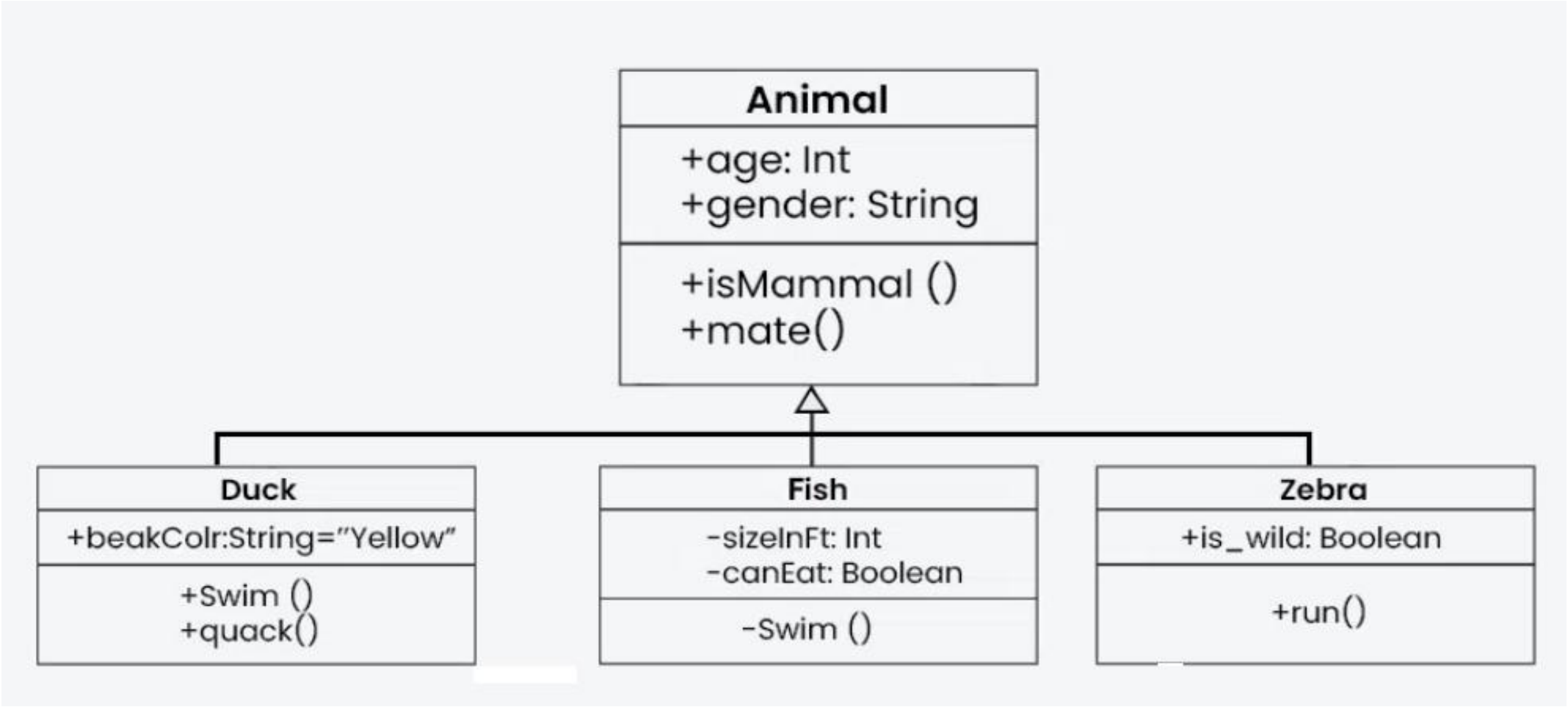


Structural UML Diagrams

- Structural UML diagrams are visual representations that depict the static aspects of a system, including its classes, objects, components, and their relationships, providing a clear view of the system's architecture.
- Structural UML diagrams include the following types:

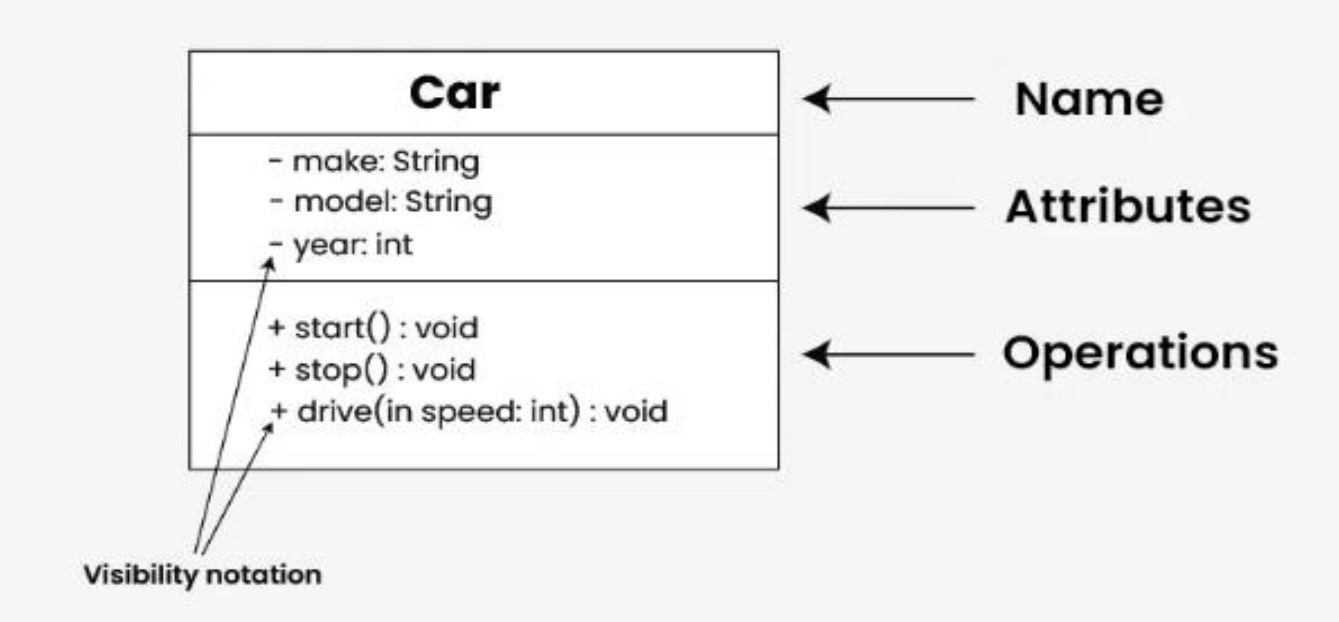
- A UML class diagram visually represents the structure of a system by showing its classes, attributes, methods, and the relationships between them.
- Helps everyone involved in a project—like developers and designers—understand how the system is organized and how its components interact.
- Helps to communicate and document the structure of the software.

UML Class Diagram example



UML Class notation

- Classes are depicted as boxes, each containing three compartments for the class name, attributes, and methods.



1. Class Name:

- The name of the class is typically written in the top compartment of the class box and is centered and bold.

2. Attributes:

- Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., public, private) and the data type of each attribute.

3. Methods:

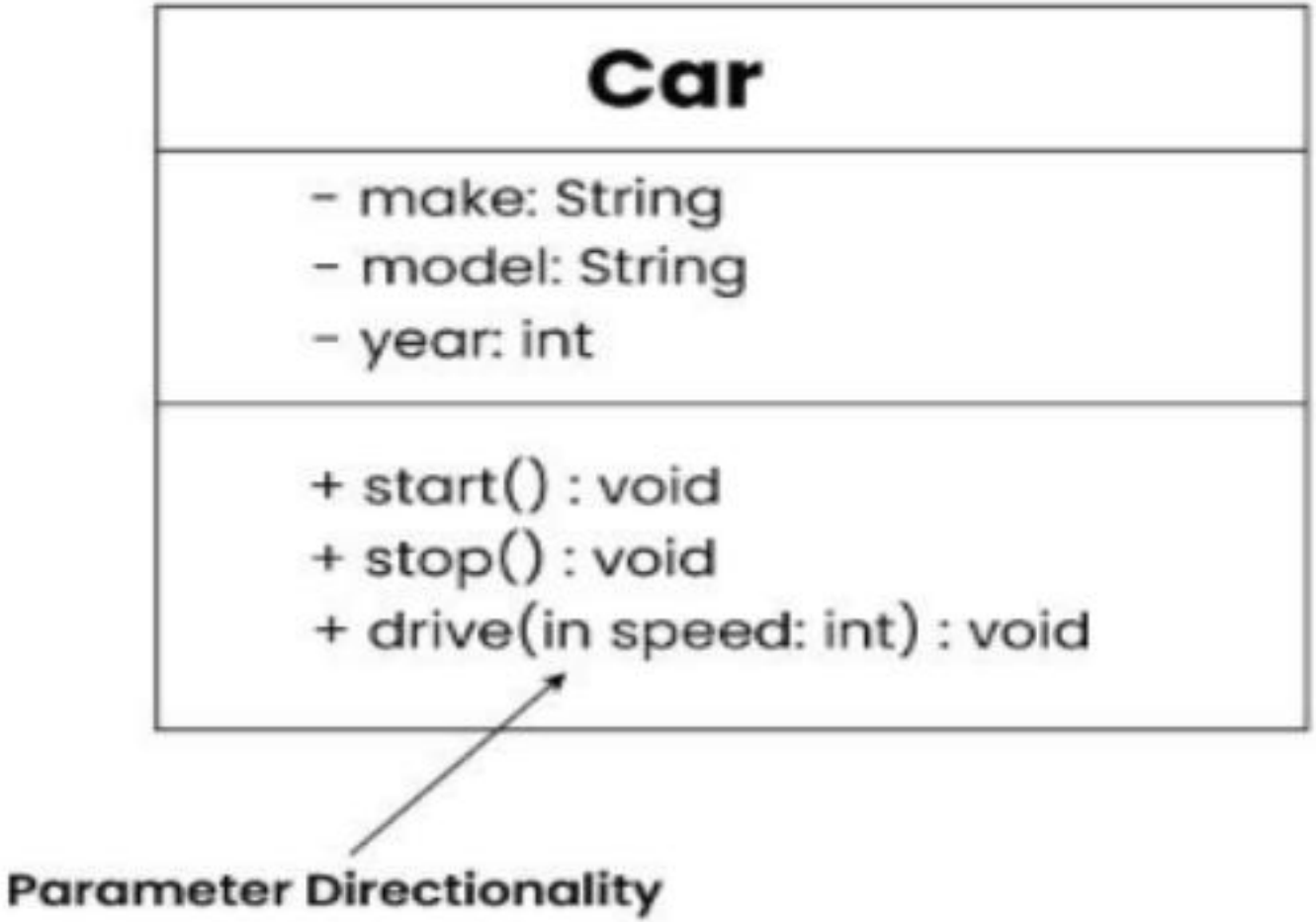
- Methods, also known as functions or operations, represent the behavior or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.

4.Visibility Notation:

- Visibility notations indicate the access level of attributes and methods. Common visibility notations include:
- + for public (visible to all classes)
- - for private (visible only within the class)
- # for protected (visible to subclasses)
- ~ for package or default visibility (visible to classes in the same package)

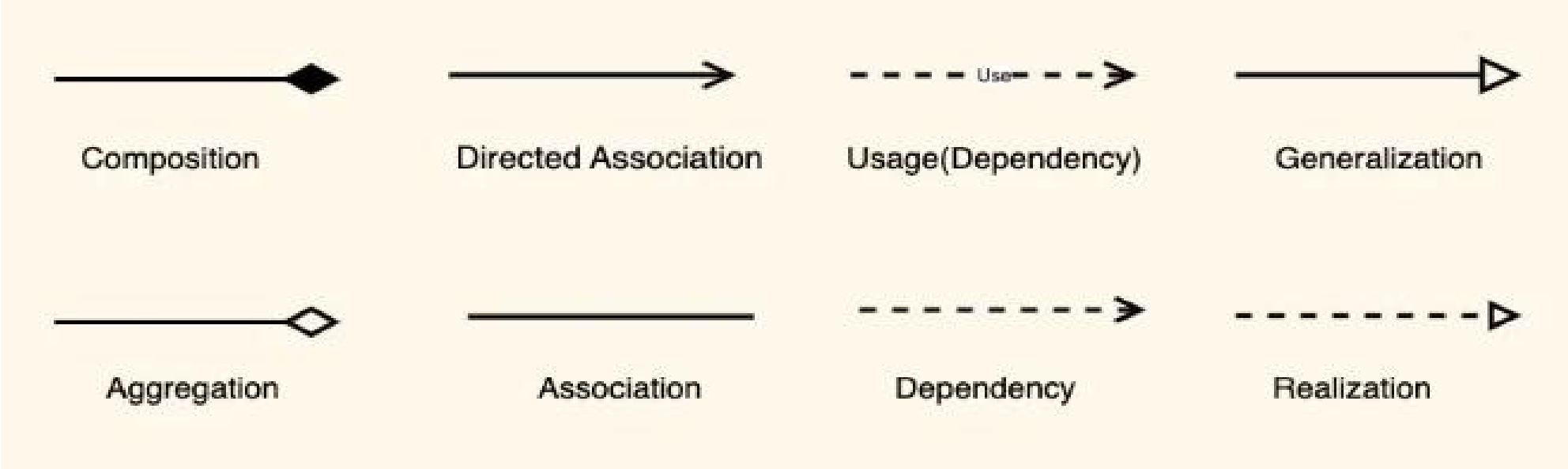
Parameter Directionality

- In class diagrams, parameter directionality refers to the indication of the flow of information between classes through method parameters.
- It helps to specify whether a parameter is an input, an output, or both. This information is crucial for understanding how data is passed between objects during method calls.


Car
- make: String
- model: String
- year: int
+ start() : void
+ stop() : void
+ drive(in speed: int) : void
Parameter Directionality

Relationships between classes

- In class diagrams, relationships between classes describe how classes are connected or interact with each other within a system. Here are some common types of relationships in class diagrams:



1. Association

- An association represents a bi-directional relationship between two classes. It indicates that instances of one class are connected to instances of another class. Associations are typically depicted as a solid line connecting the classes, with optional arrows indicating the direction of the relationship.

2. Directed Association

- A directed association in a UML class diagram represents a relationship between two classes where the association has a direction, indicating that one class is associated with another in a specific way.

3. **Aggregation**

- Aggregation is a specialized form of association that represents a "whole-part" relationship. It denotes a stronger relationship where one class (the whole) contains or is composed of another class (the part).

Aggregation is represented by a diamond shape on the side of the whole class. In this kind of relationship, the child class can exist independently of its parent class.

4. Composition

- Composition is a stronger form of aggregation, indicating a more significant ownership or dependency relationship. In composition, the part class cannot exist independently of the whole class.

Composition is represented by a filled diamond shape on the side of the whole class.

5. **Generalization(Inheritance)**

- Inheritance represents an "**is-a**" relationship between classes, where one class (the subclass or child) inherits the properties and behaviors of another class (the superclass or parent).
- **Inheritance is depicted by a solid line with a closed, hollow arrowhead pointing from the subclass to the superclass.**

6. Realization (Interface Implementation)

- Realization indicates that a class implements the features of an interface. It is often used in cases where a class realizes the operations defined by an interface. **Realization is depicted by a dashed line with an open arrowhead pointing from the implementing class to the interface.**

7. Dependency Relationship

- A dependency exists between two classes when one class relies on another, bu the relationship is not as strong as association or inheritance. It represents a mor loosely coupled connection between classes.

8. Usage(Dependency) Relationship

- A usage dependency relationship in a UML class diagram indicates that one clas (the client) utilizes or depends on another class (the supplier) to perform certai tasks or access certain functionality.
- In UML class diagrams, usage dependencies are typically represented by **dashed arrowed line pointing from the client class to the supplie class.**
- The arrow indicates the direction of the dependency, showing that the clier class depends on the services provided by the supplier class.

Benefits of Class Diagrams

- Class diagrams represent the system's classes, attributes, methods, and relationships, providing a clear view of its architecture.
- They shows various relationships between classes, such as associations and inheritance, helping stakeholders understand component connectivity.
- Class diagrams serve as a visual tool for communication among team members and stakeholders, bridging gaps between technical and non-technical audiences.
- They guide developers in coding by illustrating the design, ensuring consistency between the design and actual implementation.
- Many development tools allow for code generation from class diagrams, reducing manual errors and saving time.

Deployment Diagram in Unified Modeling Language(UML)

- A Deployment Diagram is a type of Structural UML Diagram that shows the physical deployment of software components on hardware nodes. It illustrates the mapping of software components onto the physical resources of a system, such as servers, processors, storage devices, and network infrastructure.
- They show where software components are placed on hardware devices and shows how they connect with each other. This diagram helps visualize how the software will operate across different devices

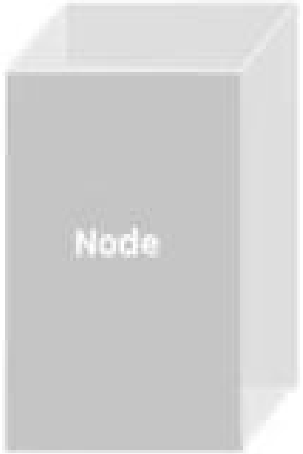
Components and Notations in Deployment Diagram

- **Nodes:** These represent the physical hardware entities where software components are deployed, such as servers, workstations, routers, etc.
- **Components:** Represent software modules or artifacts that are deployed onto nodes, including executable files, libraries, databases, and configuration files.
- **Artifacts:** Physical files that are placed on nodes represent the actual implementation of software components. These can include executable files, scripts, databases, and more.

COMPONENT

ARTIFACT

INTERFACE



NODE

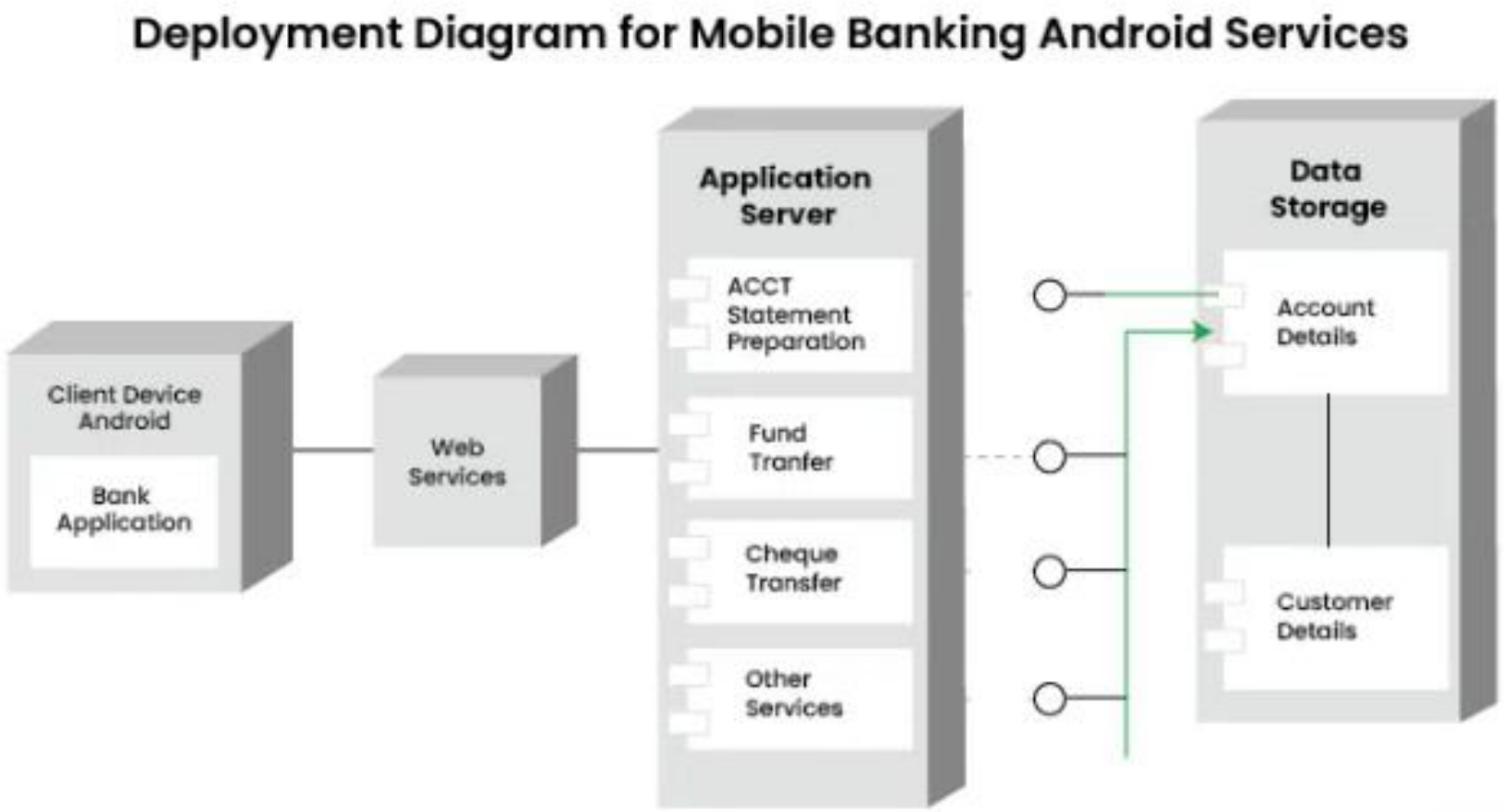
- A **component represents** a modular and reusable part of a system, typically implemented as a software module, class, or package. It encapsulates its behavior and data and can be deployed independently.
- **An artifact represents** a physical piece of information or data that is used or produced in the software development process. It can include source code files, executables, documents, libraries, configuration files, or any other item.
- **An interface** defines a contract specifying the methods or operations that a component must implement. It represents a point of interaction between different components or subsystems.
- **A node represents** a physical or computational resource, such as a hardware device, server, workstation, or computing resource, on which software components can be deployed or executed.

Steps for creating a Deployment Diagram

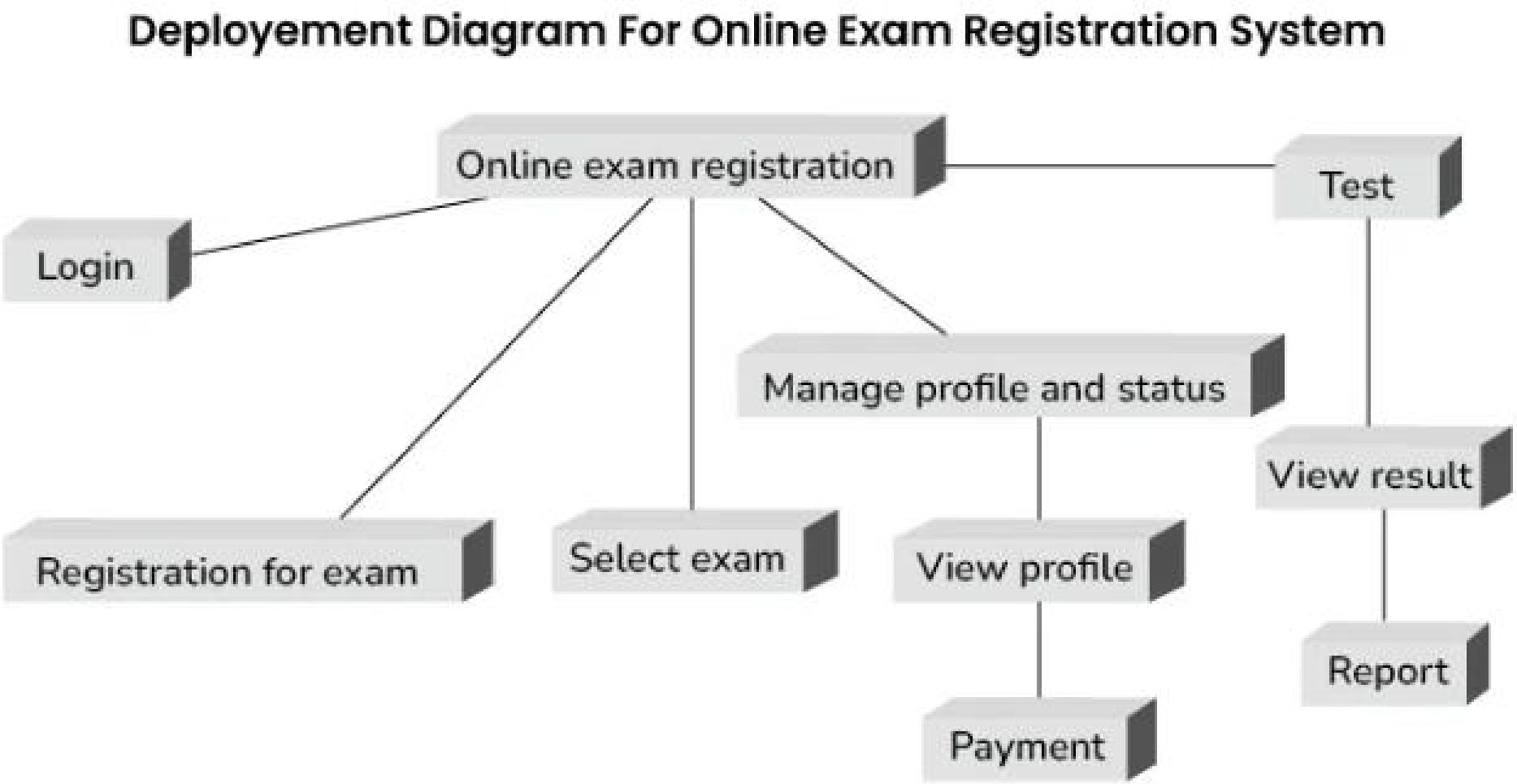
- Step1: Identify Components: List all software parts and hardware devices that will be in the deployment diagram.
- Step 2: Understand Relationships: Figure out how these parts connect and work together.
- Step 3: Gather Requirements: Collect details about hardware, network setups, and any special rules for deployment.
- Step 4: Draw Nodes and Components: Start by drawing the hardware devices (nodes) and software parts (components) using standard symbols roughly at first improvise it and draw the final one.

- Step 5: Connect Nodes and Components: Use lines or arrows to show how nodes and components are linked.
- Step 6: Add Details: Label everything clearly and include any extra info, like hardware specs or communication protocols.
- Step 7: Documentation: Write down any important decisions or assumptions made while creating the diagram.

Deployement Diagram For Mobile Banking Andorid Services. Example 1



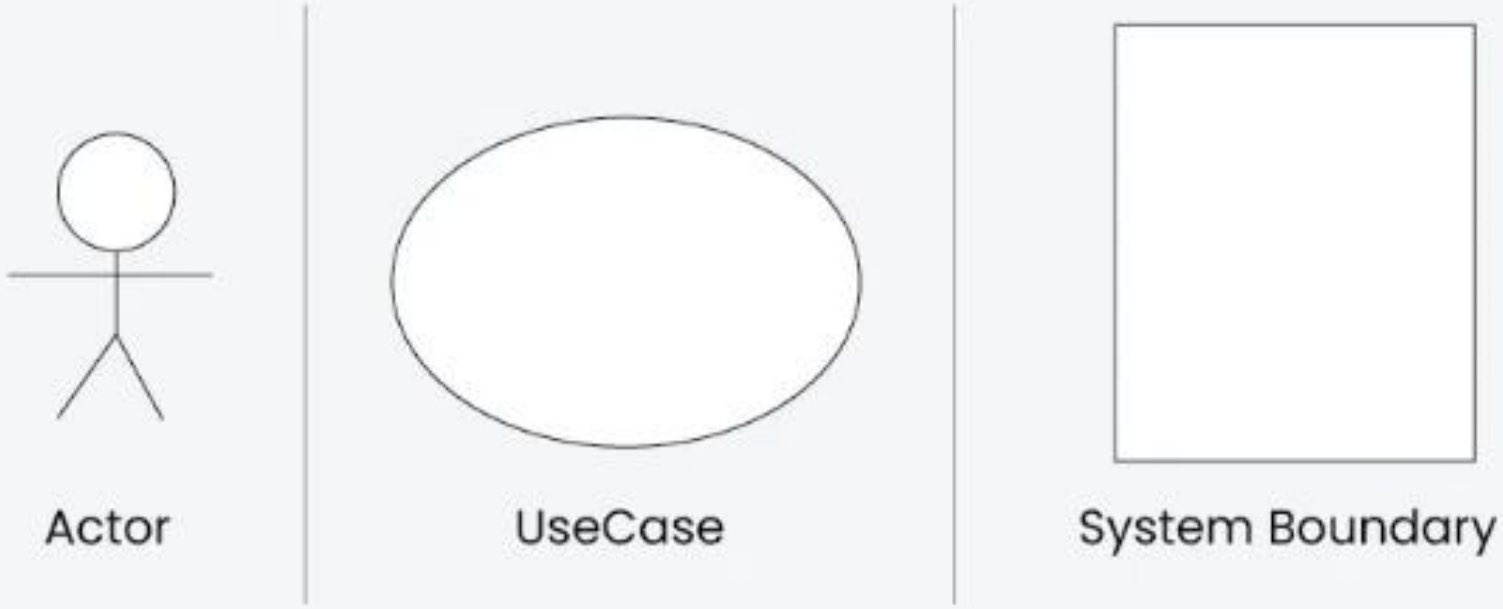
Deployement Diagram For Online Exam Registration System.



Use Case Diagram - Unified Modeling Language (UML)

- A Use Case Diagram in Unified Modeling Language (UML) is a visual representation that illustrates the interactions between users (actors) and a system.
- It captures the functional requirements of a system, showing how different users engage with various use cases, or specific functionalities, within the system.
- Use case diagrams provide a high-level overview of a system's behavior, making them useful for stakeholders, developers, and analysts to understand how a system is intended to operate from the user's perspective, and how different processes relate to one another. They are crucial for defining system scope and requirements.

Use Case Diagram Notations



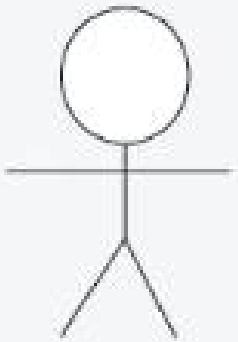
When to apply Use Case Diagram?

- When you need to gather and clarify user requirements, use case diagrams help visualize how different users interact with the system.
- If you're working with diverse groups, including non-technical stakeholders, these diagrams provide a clear and simple way to convey system functionality.
- During the system design phase, use case diagrams help outline user interactions and plan features, ensuring that the design aligns with user needs.
- When defining what is included in the system versus what is external, use case diagrams help clarify these boundaries.

1. Actors

- Actors are external entities that interact with the system. These can include users, other systems, or hardware devices.
- In the context of a Use Case Diagram, actors initiate use cases and receive the outcomes. Proper identification and understanding of actors are crucial for accurately modeling system behavior.

Actor



Actors are typically represented by stick figures

Use cases are represented by ovals.

System boundary is represented by a rectangular box.

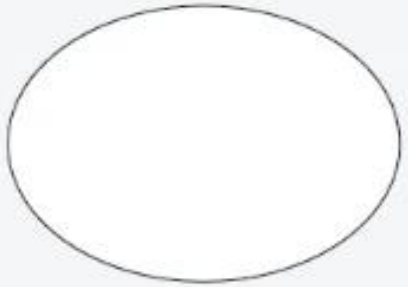
Use Case Diagrams | Unified Modeling Language (UML)

2. Use Cases

- Use cases are like scenes in the play. They represent specific things your system can do. In the online shopping system, examples of use cases could be "Place Order," "Track Delivery," or "Update Product Information". Use cases are represented by ovals.

Usecase

Actors are typically represented by stick figures



Use cases are represented by ovals.

System boundary is represented by a rectangular box.

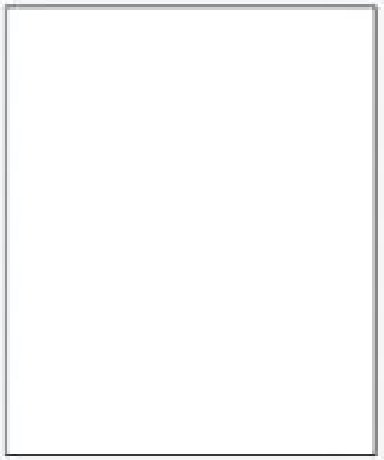
3. System Boundary

- The system boundary is a visual representation of the scope or limits of the system you are modeling. It defines what is inside the system and what is outside.
- The boundary helps to establish a clear distinction between the elements that are part of the system and those that are external to it. The system boundary is typically represented by a rectangular box that surrounds all the use cases of the system.

System

Actors are typically represented by stick figures

Use cases are represented by ovals.



System boundary is represented by a rectangular box.

Use Case Diagram Relationships

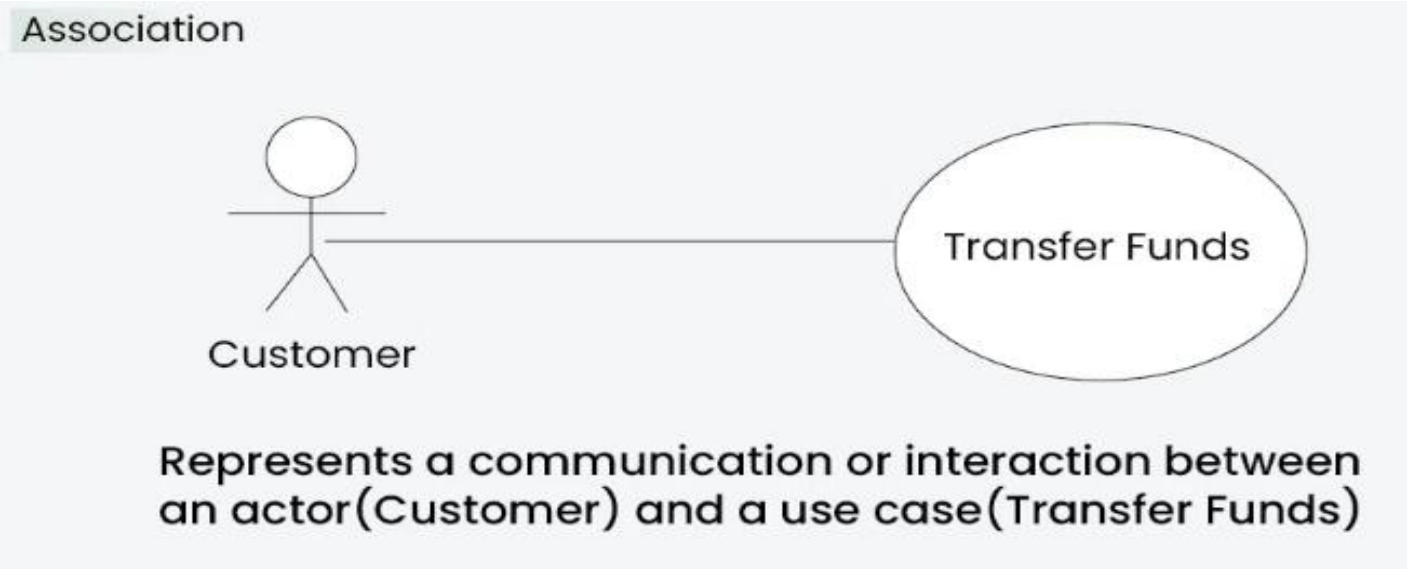
- In a Use Case Diagram, relationships play a crucial role in depicting the interactions between actors and use cases.
- These relationships provide a comprehensive view of the system's functionality and its various scenarios. Let's delve into the key types of relationships and explore examples to illustrate their usage.

1. Association Relationship

- The Association Relationship represents a communication or interaction between an actor and a use case. It is depicted by a line connecting the actor to the use case.
- This relationship signifies that the actor is involved in the functionality described by the use case.

Example: Online Banking System

- **Actor:** Customer
- **Use Case**: Transfer Funds
- **Association:** A line connecting the "Customer" actor to the "Transfer Funds" use case, indicating the customer's involvement in the funds transfer process.

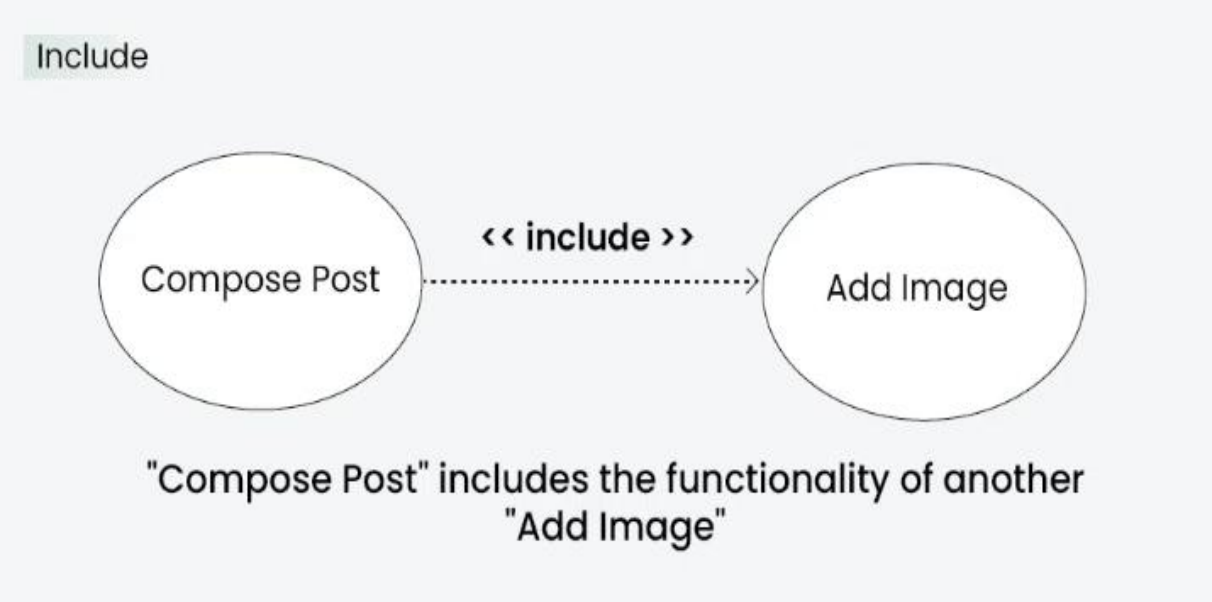


2. Include Relationship

- The Include Relationship indicates that a use case includes the functionality of another use case.
- It is denoted by a dashed arrow pointing from the including use case to the included use case.
- This relationship promotes modular and reusable design.

Example: Social Media Posting

- Use Cases: Compose Post, Add Image
- Include Relationship: The "Compose Post" use case includes the functionality of "Add Image." Therefore, composing a post includes the action of adding an image.

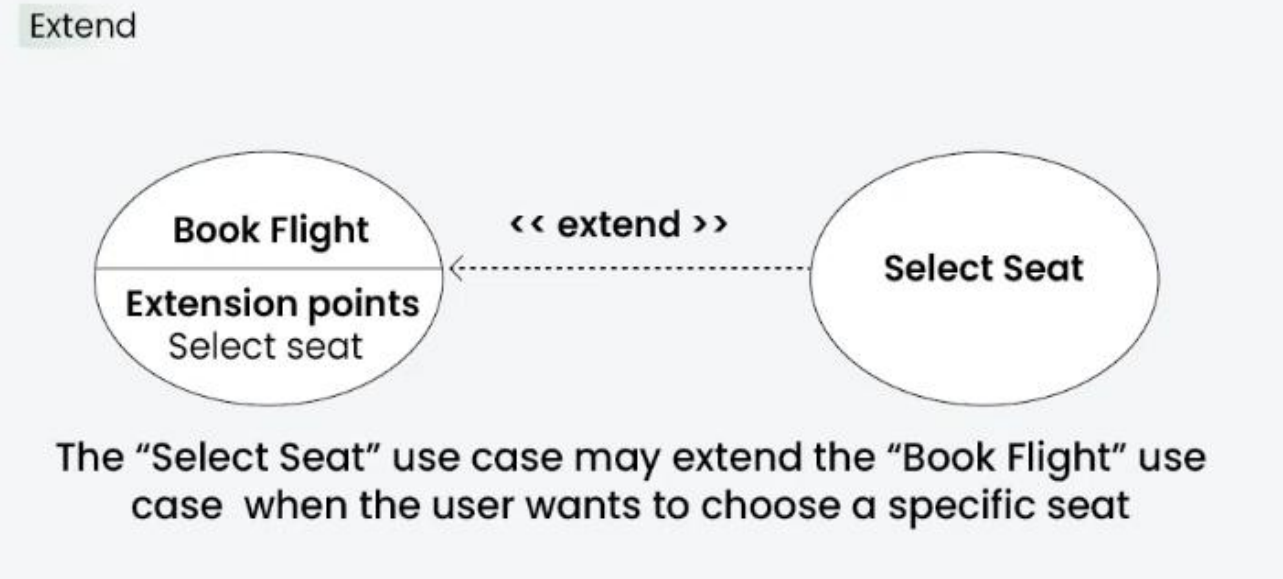


3. Extend Relationship

- The Extend Relationship illustrates that a use case can be extended by another use case under specific conditions. It is represented by a dashed arrow with the keyword "**extend**."
- This relationship is useful for handling optional or exceptional behavior.

Example: Flight Booking System

- **Use Cases:** Book Flight, Select Seat
- **Extend Relationship:** The "Select Seat" use case may extend the "Book Flight" use case when the user wants to choose a specific seat, but it is an optional step.

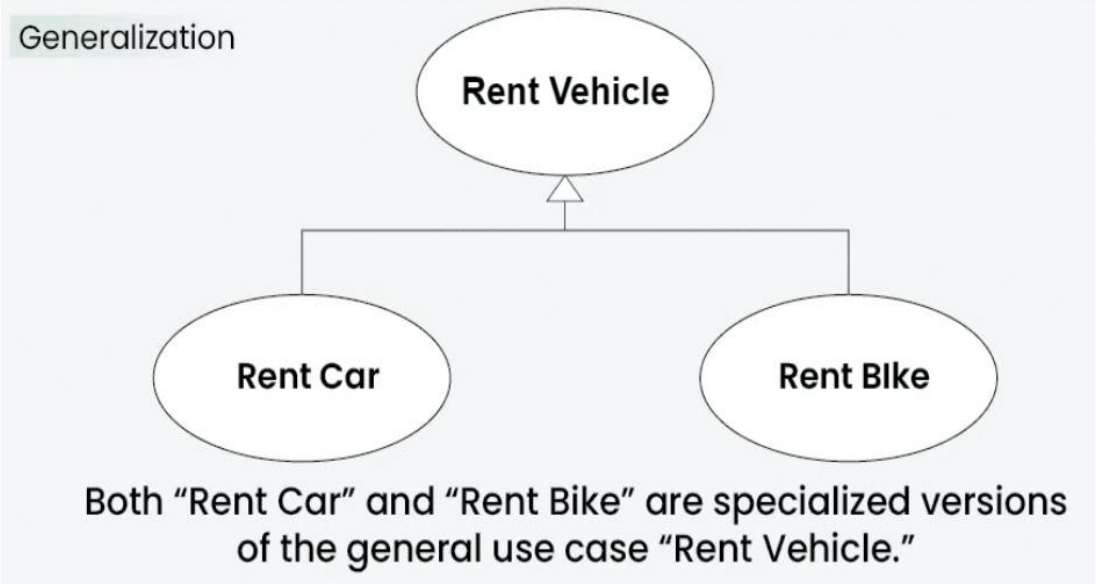


4. Generalization Relationship

- The Generalization Relationship establishes an "is-a" connection between two use cases, indicating that one use case is a specialized version of another. It is represented by an arrow pointing from the specialized use case to the general use case.

Example: Vehicle Rental System

- **Use Cases:** Rent Car, Rent Bike
- **Generalization Relationship**: Both "Rent Car" and "Rent Bike" are specialized versions of the general use case "Rent Vehicle."



Use Case Diagram example(Online Shopping System)

Actors:

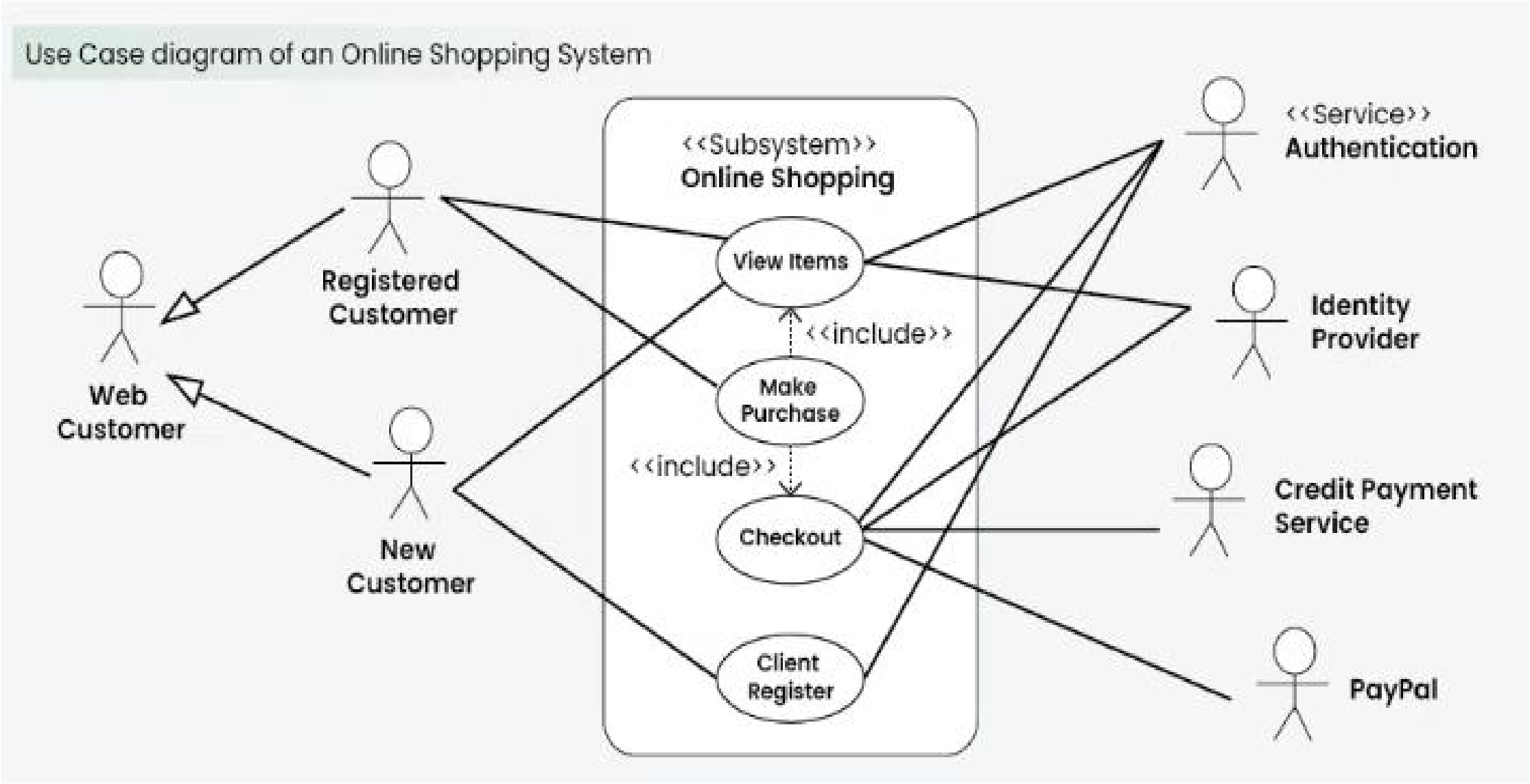
- Customer
- Admin

Use Cases:

- Browse Products
- Add to Cart
- Checkout
- Manage Inventory (Admin)

Relations:

- The Customer can browse products, add to the cart, and complete the checkout.
- The Admin can manage the inventory.


Use Case diagram of an Online Shopping System
<<Subsystem>>
Online Shopping
View Items
<<include>>
Make Purchase
<<include>>
Checkout
Client Register
Registered Customer
Web Customer
New Customer
<<Service>>
Authentication
Identity Provider
Credit Payment Service
PayPal

Sequence Diagrams - Unified Modeling Language (UML)

- A Sequence Diagram is a key component of Unified Modeling Language (UML) used to visualize the interaction between objects in a sequential order. It focuses on how objects communicate with each other over time, making it an essential tool for modeling dynamic behavior in a system. Sequence diagrams illustrate object interactions, message flows, and the sequence of operations, making them valuable for understanding use cases, designing system architecture, and documenting complex processes.

Sequence Diagrams

Sequence Diagram Notations

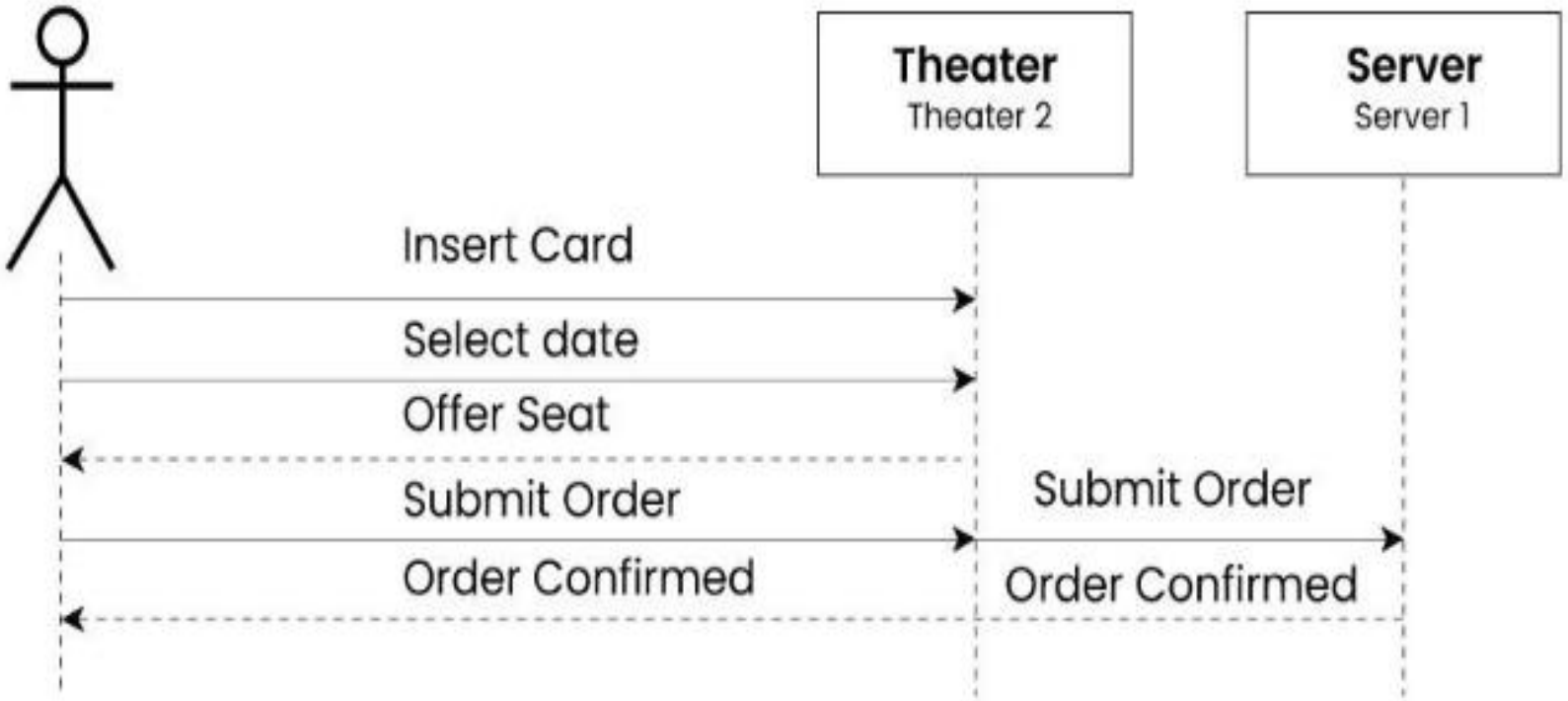
1. Actors

- An actor in a UML diagram represents a type of role where it interacts with the system and its objects. It is important to note here that an actor is always outside the scope of the system we aim to model using the UML diagram.

Notation symbol for actor

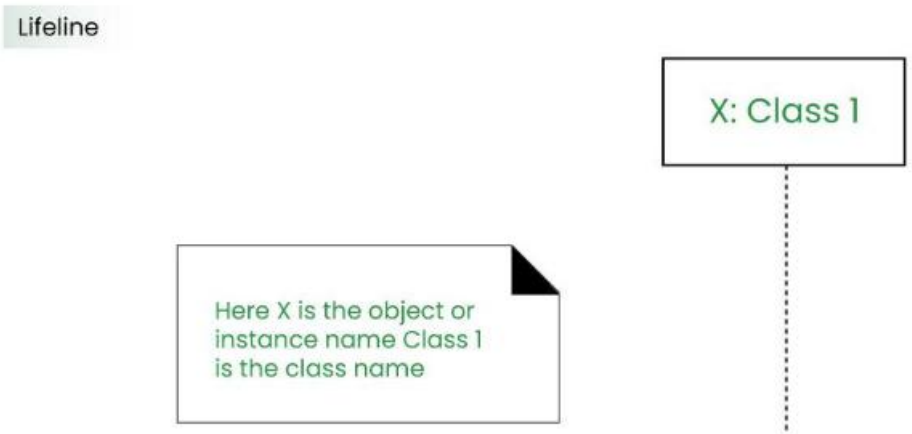
For example:

User interacting with seat reservation system



2. Lifelines

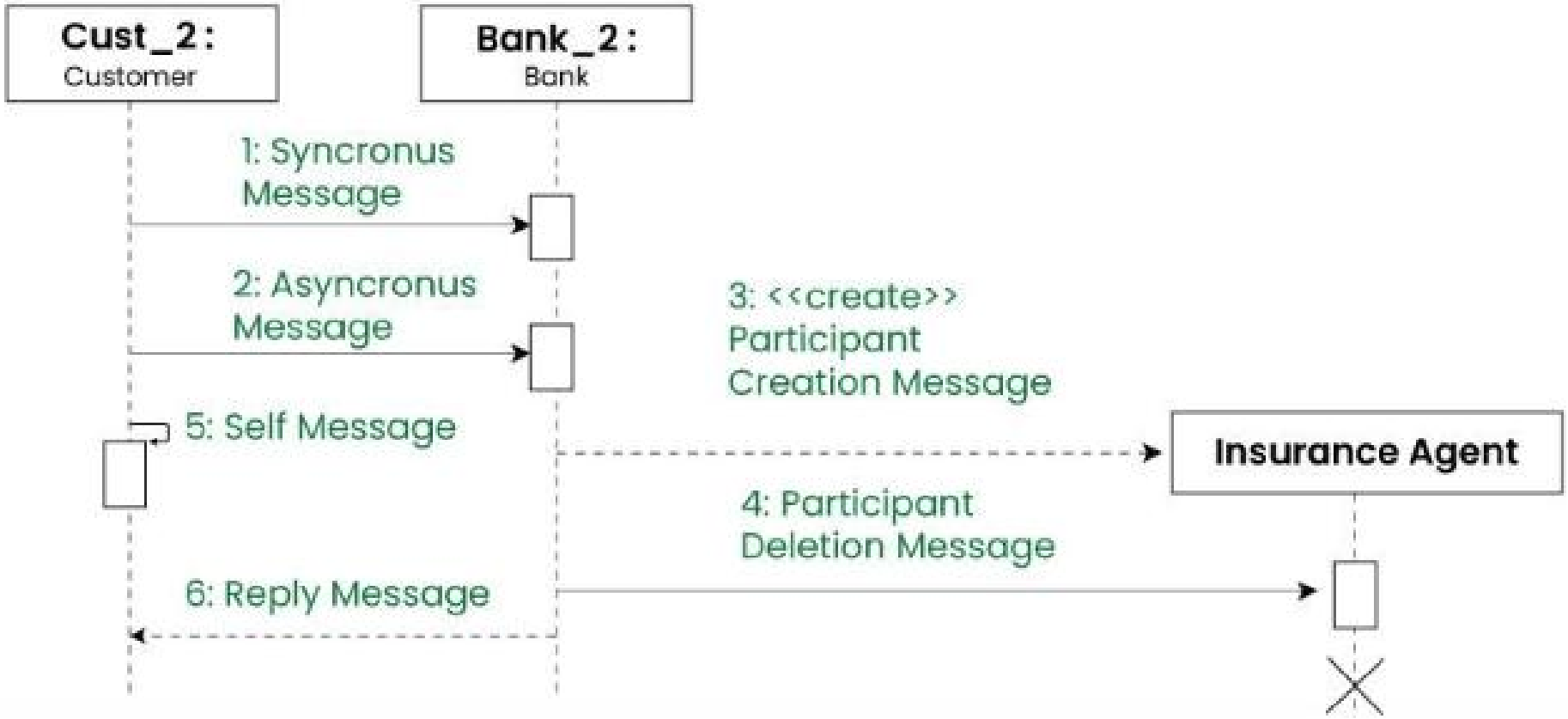
- A lifeline is a named element which depicts an individual participant in a sequence diagram. So basically each instance in a sequence diagram is represented by a lifeline. Lifeline elements are located at the top in a sequence diagram. The standard in UML for naming a lifeline follows the following format:
- Instance Name : Class Name



3. Messages

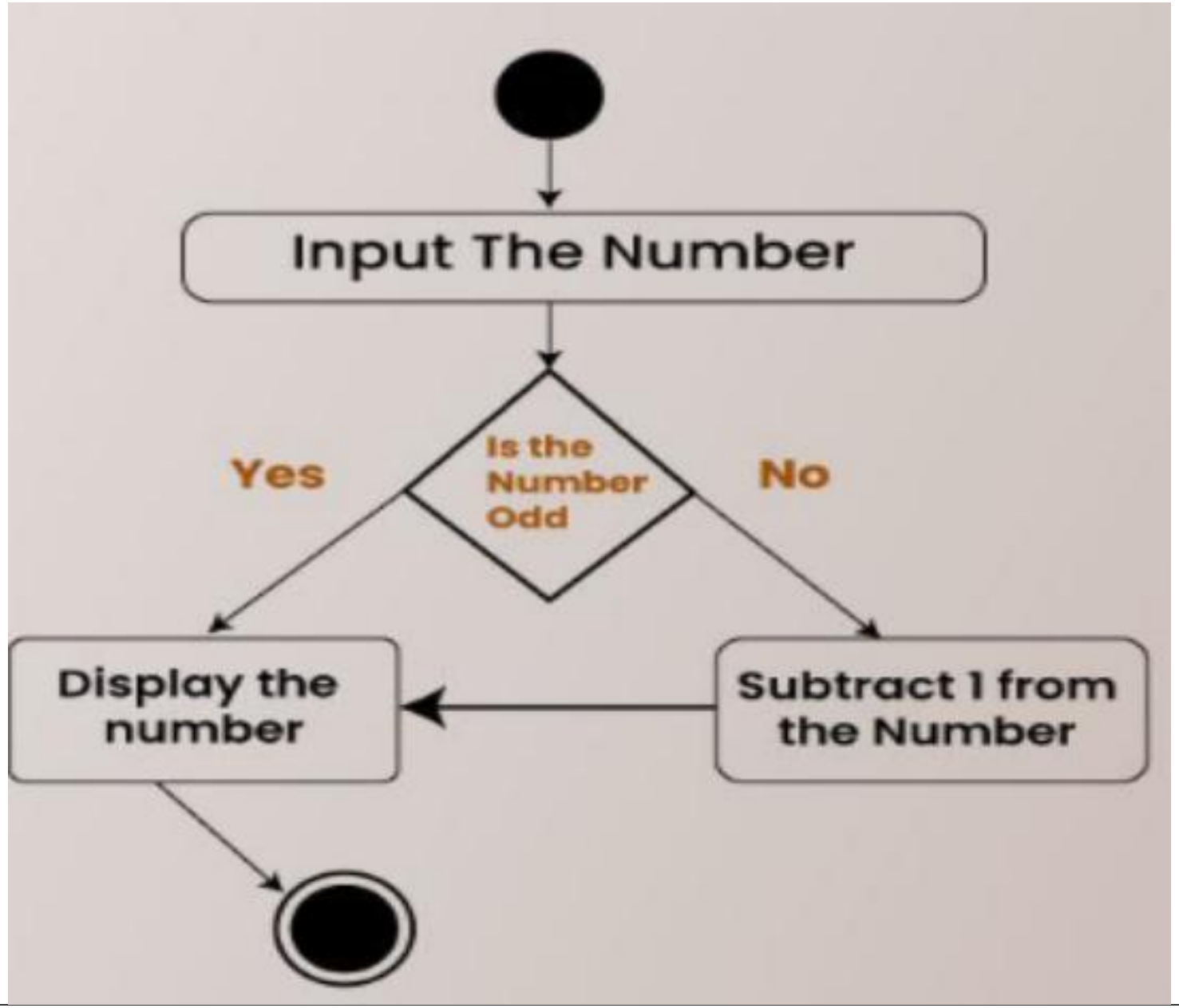
- Communication between objects is depicted using messages. The messages appear in a sequential order on the lifeline.
- We represent messages using arrows.
- Lifelines and messages form the core of a sequence diagram.

Different Types of Messages



Activity Diagrams - Unified Modeling Language (UML)

- Activity diagrams are an essential part of the Unified Modeling Language (UML) that help visualize workflows, processes, or activities within a system.
- They depict how different actions are connected and how a system moves from one state to another. By offering a clear picture of both simple and complex workflows, activity diagrams make it easier for developers and stakeholders to understand how various elements interact in a system.


Input The Number
Is the Number Odd
Yes
No
Display the number
Subtract 1 from the Number